

b) Set root as root's left child.

2.2 Pop an item from stack and set it as root.

a) If the popped item has a right child and the right child is at top of stack, then remove the right child from stack, push the root back and set root as root's right child.

1a) Else print root's data and set root as NULL.

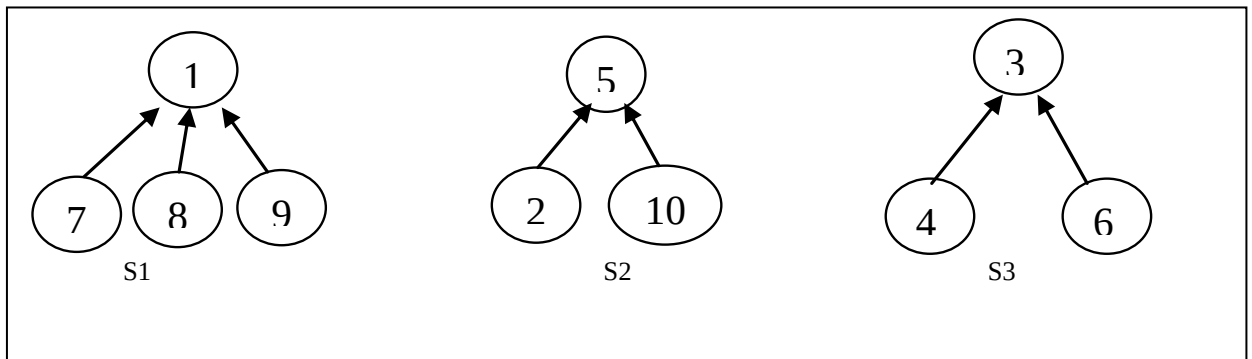
2.3 Repeat steps 2.1 and 2.2 while stack is not empty.

Disjoint Sets: If S_i and S_j , $i \neq j$ are two sets, then there is no element that is in both S_i and S_j .

For example: $n=10$ elements can be partitioned into three disjoint sets,

$S_1 = \{1, 7, 8, 9\}$
 $S_2 = \{2, 5, 10\}$
 $S_3 = \{3, 4, 6\}$

Tree representation of sets:

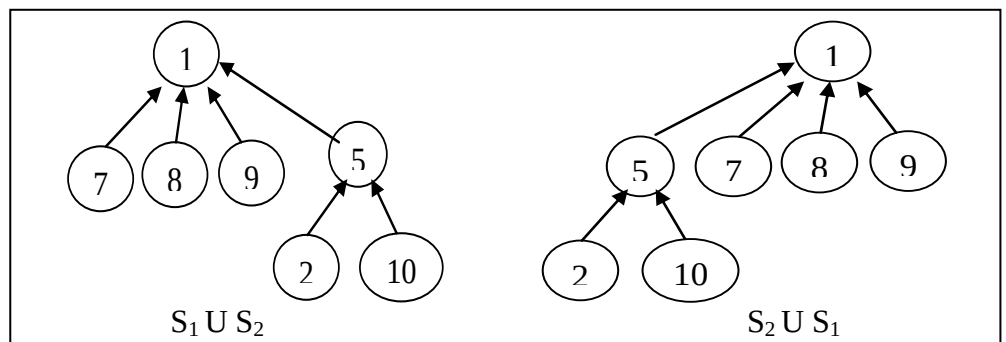


Disjoint set Operations:

- Disjoint set Union
- Find(i)

Disjoint set Union: Means Combination of two disjoint sets elements. Form above example $S_1 \cup S_2 = \{1, 7, 8, 9, 5, 2, 10\}$

For $S_1 \cup S_2$ tree representation, simply make one of the tree is a subtree of the other.



Find: Given element i , find the set containing i .

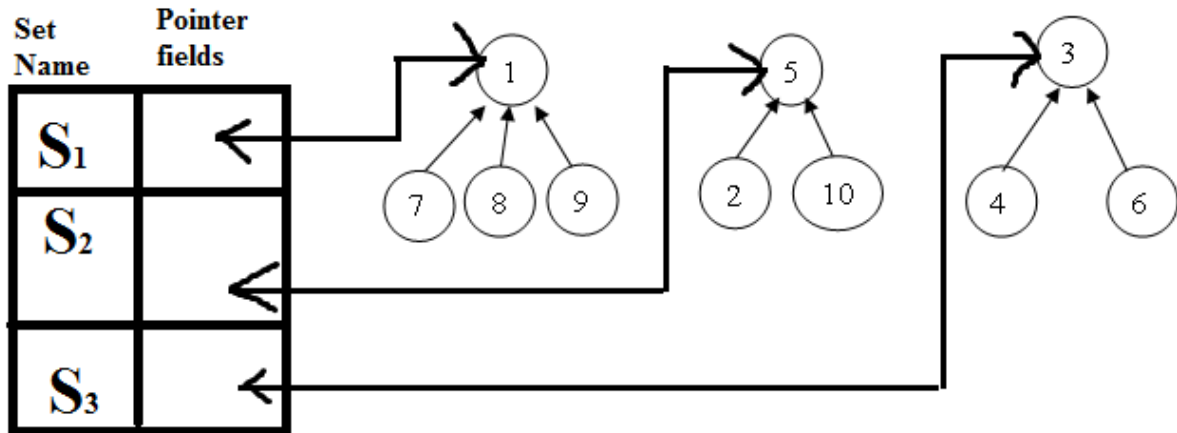
Form above example:

$\text{Find}(4) \rightarrow S_3$

Find(1)→S₁
Find(10)→S₂

Data representation of sets:

Tress can be accomplished easily if, with each set name, we keep a pointer to the root of the tree representing that set.



For presenting the union and find algorithms, we ignore the set names and identify sets just by the roots of the trees representing them.

For example: if we determine that element 'i' is in a tree with root 'j' has a pointer to entry 'k' in the set name table, then the set name is just **name[k]**

For unite (**adding or combine**) to a particular set we use FindPointer function.

Example: If you wish to unite to S_i and S_j then we wish to unite the tree with roots

FindPointer (S_i) and FindPointer (S_j)

FindPointer→ is a function that takes a set name and determines the root of the tree that represents it.

For determining operations:

Find(i)→ 1st determine the root of the tree and find its pointer to entry in setname table.

Union(i, j)→ Means union of two trees whose roots are i and j.

If set contains numbers 1 through n, we represents tree node

P[1:n].

n→Maximum number of elements.

Each node represent in array

i	1	2	3	4	5	6	7	8	9	10
P	-1	5	-1	3	-1	3	1	1	1	5

Find(i) by following the indices, starting at i until we reach a node with parent value -1.

Example: Find(6) start at 6 and then moves to 6's parent. Since P[3] is negative, we reached the root.

Algorithm for finding Union(i, j):	Algorithm for find(i)
Algorithm Simple union(i, j) { P[i]:=j; // Accomplishes the union }	Algorithm SimpleFind(i) { While(P[i]≥0) do i:=P[i]; return i; }

If n numbers of roots are there then the above algorithms are not useful for union and find.

For union of n trees → Union(1,2), Union(2,3), Union(3,4),.....Union(n-1,n).

For Find i in n trees → Find(1), Find(2),....Find(n).

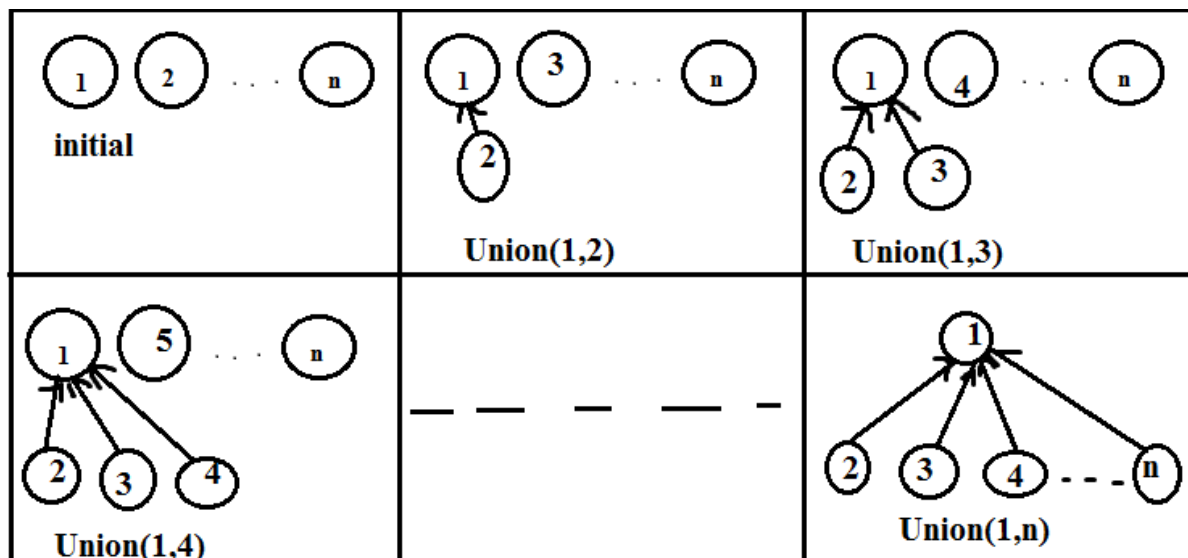
Time taken for the union (simple union) is → O(1) (constant).
For the n-1 unions → O(n).

Time taken for the find for an element at level i of a tree is → O(i).
For n finds → O(n²).

To improve the performance of our union and find algorithms by avoiding the creation of degenerate trees. For this we use a weighting rule for union(i, j)

Weighting rule for Union(i, j):

If the number of nodes in the tree with root 'i' is less than the tree with root 'j', then make 'j' the parent of 'i'; otherwise make 'i' the parent of 'j'.



Tree obtained using the weighting rule

Algorithm for weightedUnion(i, j)

```
Algorithm WeightedUnion(i,j)
//Union sets with roots i and j, i≠j
// The weighting rule, p[i]= -count[i] and p[j]= -count[j].
{
temp := p[i]+p[j];
if (p[i]>p[j]) then
{ // i has fewer nodes.
P[i]:=j;
P[j]:=temp;
}
else
{ // j has fewer or equal nodes.
P[j] := i;
P[i] := temp;
}
}
```

For implementing the weighting rule, we need to know how many nodes there are in every tree.

For this we maintain a count field in the root of every tree.

$i \rightarrow$ root node

$\text{count}[i] \rightarrow$ number of nodes in the tree.

Time required for this above algorithm is $O(1)$ + time for remaining unchanged is determined by using **Lemma**.

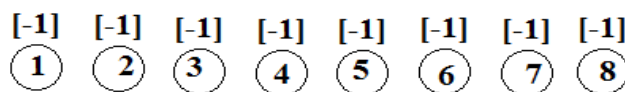
Lemma:- Let T be a tree with **m** nodes created as a result of a sequence of unions each performed using WeightedUnion. The height of T is no greater than $\lceil \log_2 m \rceil + 1$.

Collapsing rule: If 'j' is a node on the path from 'i' to its root and $p[i] \neq \text{root}[i]$, then set $p[j]$ to $\text{root}[i]$.

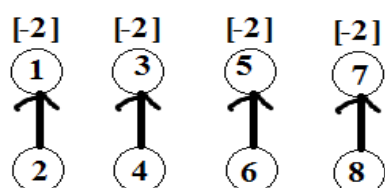
Algorithm for Collapsing find.
<pre> Algorithm CollapsingFind(i) //Find the root of the tree containing element i. //collapsing rule to collapse all nodes from i to the root. { r:=i; while(p[r]>0) do r := p[r]; //Find the root. While(i ≠ r) do // Collapse nodes from i to root r. { s:=p[i]; p[i]:=r; i:=s; } return r; } </pre>

Collapsing find algorithm is used to perform find operation on the tree created by WeightedUnion.

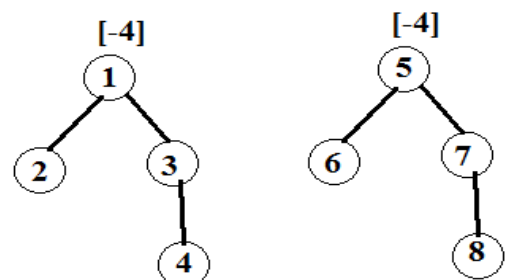
For example: Tree created by using WeightedUnion



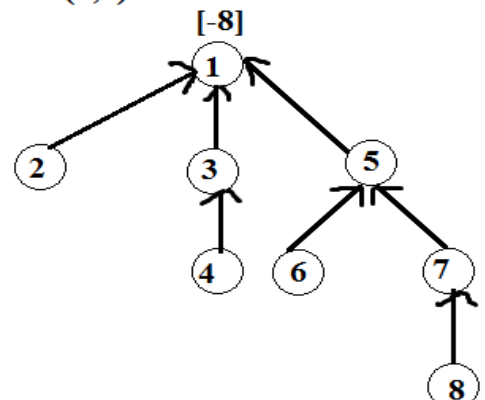
(a) initial height -1 tree



(b) Height -2 trees following Union(1,2),(3,4),(5,6),(7,8)



(c) Height -3 trees following Union (1,3) and (5,7)



(d) Height -4 tree Following Union(1,5)

Now process the following eight finds: Find(8), Find(8),.....Find(8)

If SimpleFind is used, each Find(8) requires going up three parent link fields for a total of 24 moves to process all eight finds.